

4. Fisheries production models

Beth Babcock

2025-09-02

```
library(ggplot2)
theme_set(theme_bw())
```

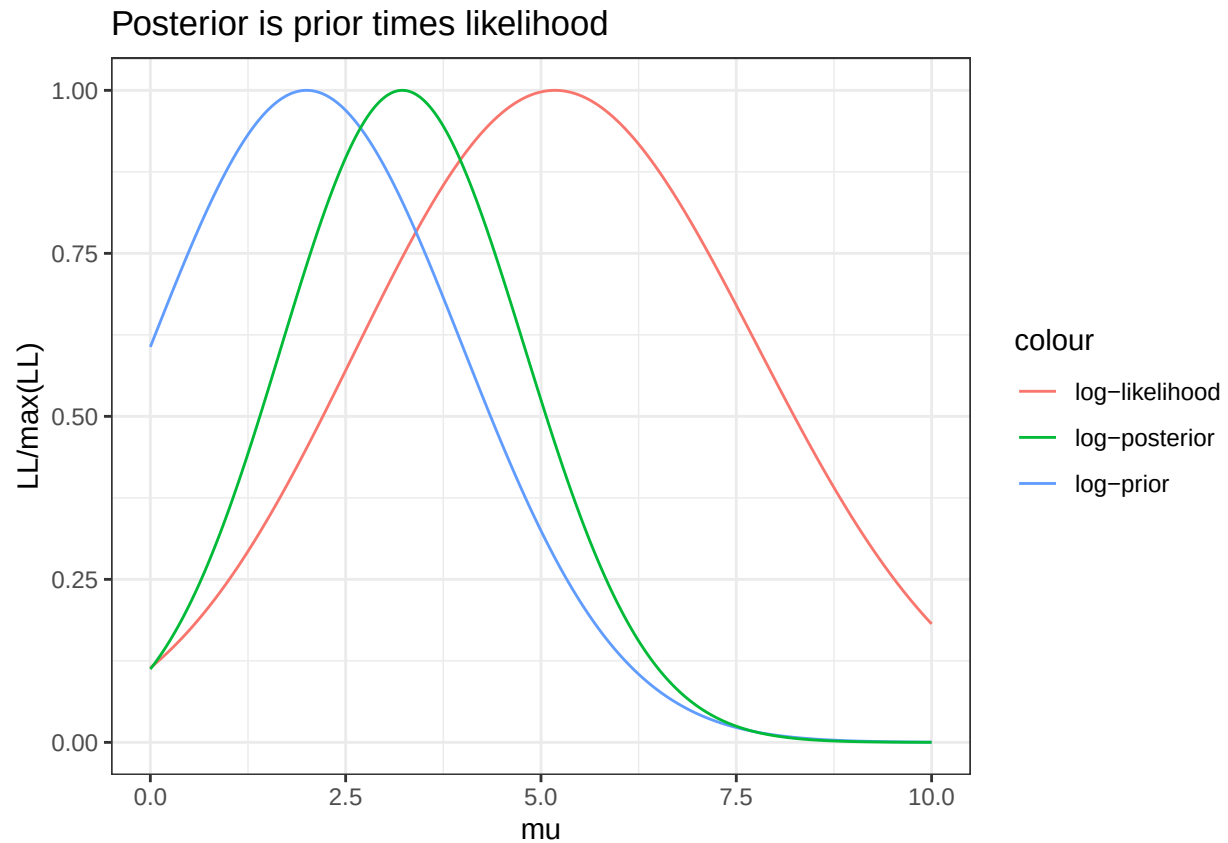
Fitting a model with Bayesian methods

This is the same data we looked at for a normal likelihood.

```
data<-data.frame(i=1:30,x=c(7.72, 4.79, 5.78, 4.89, 2.25, 4.17, 4.21, 4.88, 7.2, 6.53, 4.67, 4.49, 6.39,
3.78, 5.68, 2.74, 7.87, 8.96, 4.27, 2.91, 6.14, 4.73))
```

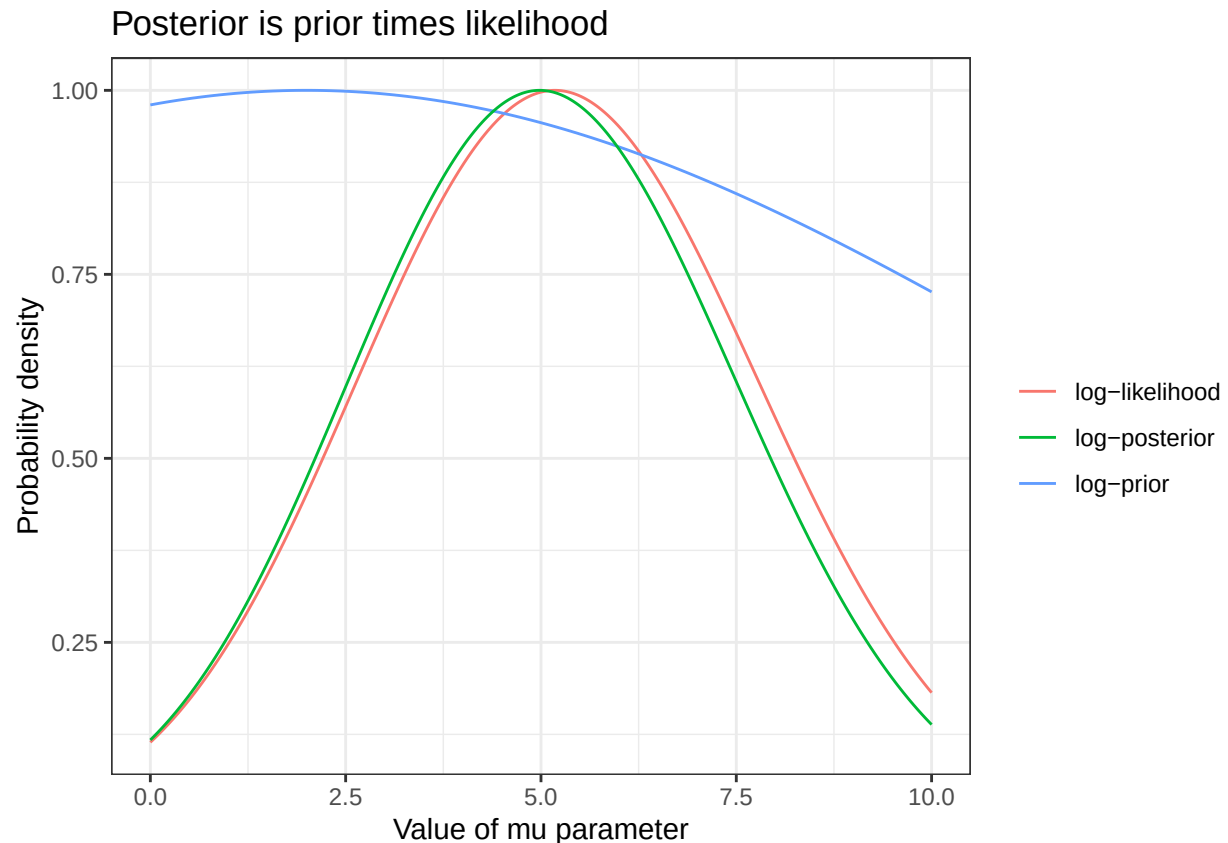
Now let's try some priors. First, an informative prior. Let's assume that we have good information about the value of μ from another study, so our prior has a mean of 2 and sd of 0.2.

```
priormean<-2
priorsd<-2
posterior<-data.frame(mu=seq(0,10,0.01))
for(i in 1:nrow(posterior)) {
  posterior$LL[i]<-sum(dnorm(data$x,mean=posterior$mu[i],sd=2))
}
posterior$logprior<-dnorm(posterior$mu,mean=priormean,sd=priorsd)
posterior$logposterior<-posterior$LL*posterior$logprior
ggplot(posterior,aes(x=mu))+
  geom_line(aes(y=LL/max(LL),color="log-likelihood"))+
  geom_line(aes(y=logprior/max(logprior),color="log-prior"))+
  geom_line(aes(y=logposterior/max(logposterior),color="log-posterior"))+
  ggtitle("Posterior is prior times likelihood")
```



Same with a less informative prior

```
priormean<-2
priorsd<-10
posterior<-data.frame(mu=seq(0,10,0.01))
for(i in 1:nrow(posterior)) {
  posterior$LL[i]<-sum(dnorm(data$x,mean=posterior$mu[i],sd=2))
}
posterior$logprior<-dnorm(posterior$mu,mean=priormean,sd=priorsd)
posterior$logposterior<-posterior$LL*posterior$logprior
ggplot(posterior,aes(x=mu))+
  geom_line(aes(y=LL/max(LL),color="log-likelihood"))+
  geom_line(aes(y=logprior/max(logprior),color="log-prior"))+
  geom_line(aes(y=logposterior/max(logposterior),color="log-posterior"))+
  ggtitle("Posterior is prior times likelihood")+
  labs(y="Probability density",x="Value of mu parameter",color="")
```



Posterior is more similar to the likelihood

Now make a function to minimize to get the mode of the posterior distribution

```
getNegLogPosterior<-function(theta,data) {
  LL<-dnorm(x=data$x, #data is an input, with a variable called x
            mean=theta["mu"], #mean is the parameter we want to calculate
            sd=2,
            log=TRUE) #fix standard deviation for this example
  logprior<-dnorm(theta["mu"],data$priormean,data$priorsd,log=TRUE)
  -sum(LL)-logprior #return negative total log likelihood plus priors
}
getNegLogPosterior(c(mu=5.2),list(x=data$x,priormean=2,priorsd=2))
```

```
##      mu
## 60.44321
```

Use minimizer function optim to find the value of mu that has the lowest -logposterior

```
modFit<-optim(par=c(mu=3),
              fn=getNegLogPosterior,
              data=list(x=data$x,priormean=2,priorsd=2),
              method="L-BFGS-B")
modFit$par
```

```
##      mu
## 5.160645
```

Bayesian production models in R.

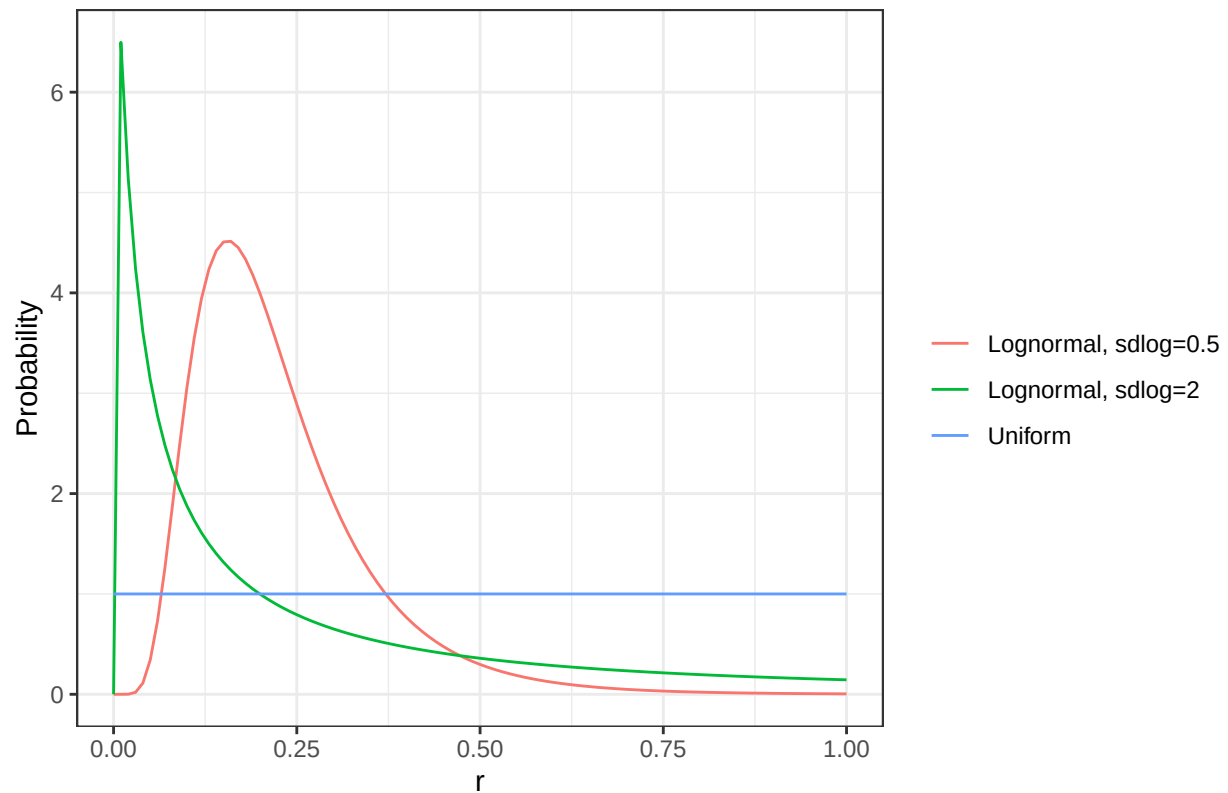
Possible priors

```
dlogunif<-function(x,minval,maxval,logTrans=TRUE) {  
  if(minval==0) minval<-0.0000001  
  prob<-1/(x*(log(maxval)-log(minval)))  
  if(logTrans) return<-log(prob) else  
    return=prob  
  return  
}  
dlogunif(1,0,1000,logTrans=FALSE)
```

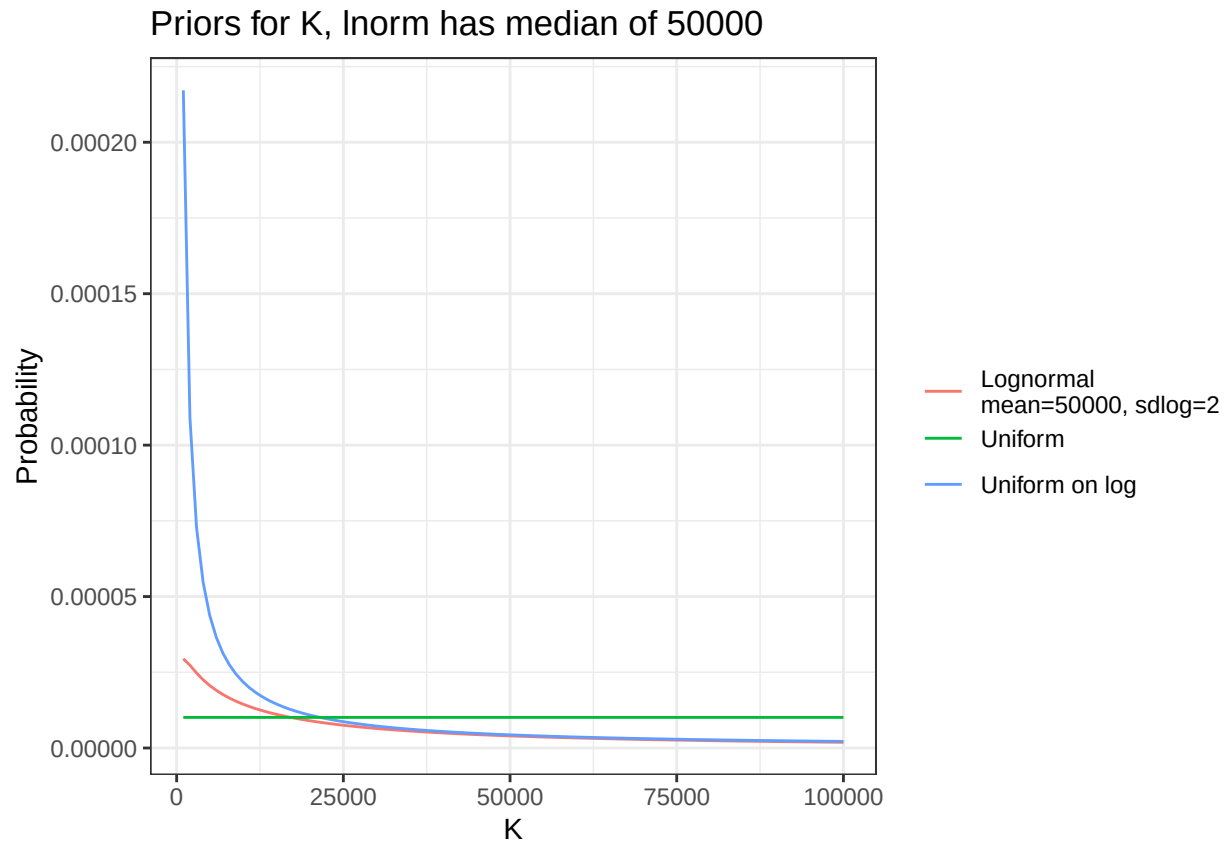
```
## [1] 0.04342945
```

```
ggplot(data.frame(r=c(0,1)),aes(x=r))+  
  geom_function(aes(color="Lognormal, sdlog=0.5"),  
    fun=dlnorm,  
    args=list(meanlog = log(0.2), sdlog = 0.5))+  
  geom_function(aes(color="Lognormal, sdlog=2"),  
    fun=dlnorm,  
    args=list(meanlog = log(0.2), sdlog = 2))+  
  geom_function(aes(color="Uniform"),  
    fun=dunif,  
    args=list(min=0,max=1))+  
  ggtitle("Priors for r with median of 0.2")+  
  labs(x="r",y="Probability",color="")
```

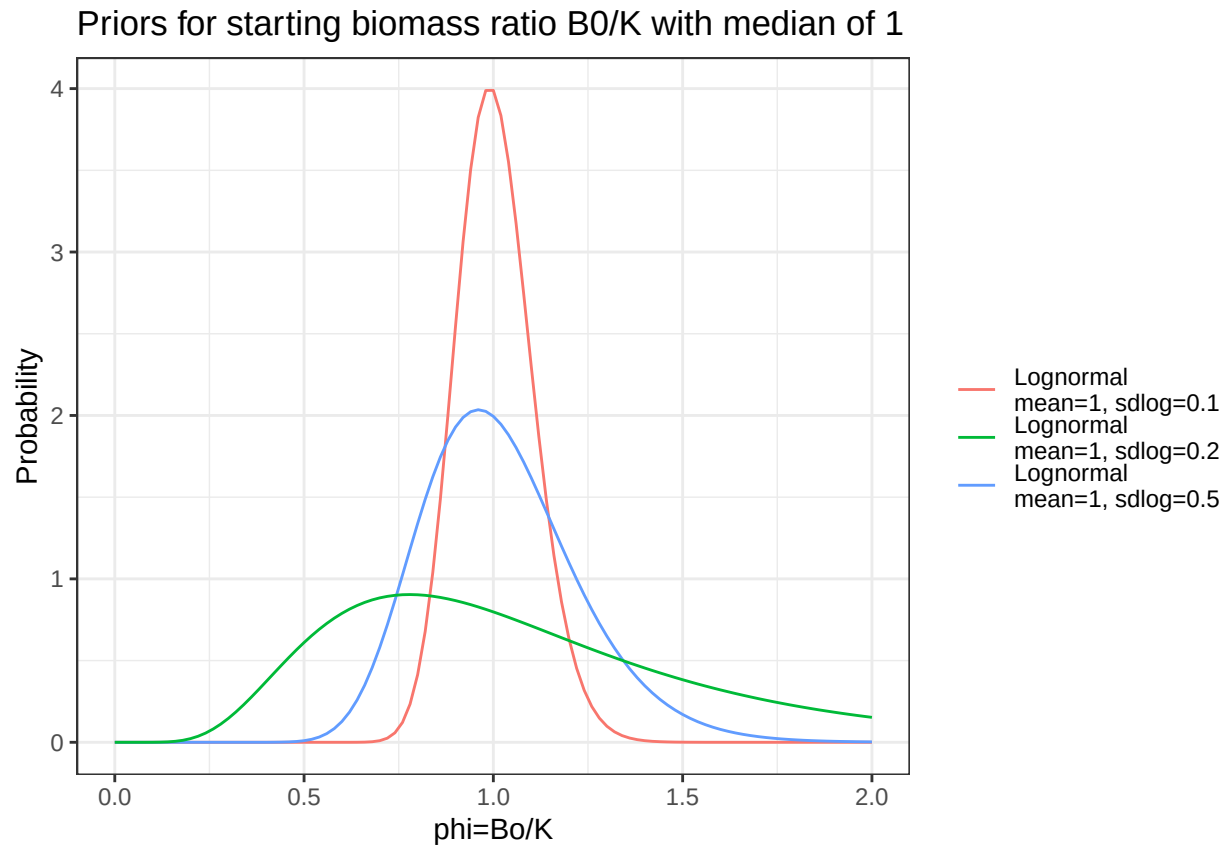
Priors for r with median of 0.2



```
ggplot(data.frame(K=c(1000,100000)),aes(x=K))+
  geom_function(aes(color="Lognormal\nmean=50000, sdlog=2"),
    fun=dlnorm,
    args=list(meanlog = log(50000), sdlog = 2))+
  geom_function(aes(color="Uniform on log"),
    fun=dlogunif,
    args=list(minval=1000,maxval=100000,logTrans=FALSE))+
  geom_function(aes(color="Uniform"),
    fun=dunif,
    args=list(min=1000,max=100000))+
  ggtitle("Priors for K, lnorm has median of 50000")+
  labs(x="K",y="Probability",color="")
```



```
ggplot(data.frame(phi=c(0,2)),aes(x=phi))+
  geom_function(aes(color="Lognormal\nmean=1, sdlog=0.1"),
    fun=dlnorm,
    args=list(meanlog = log(1), sdlog = 0.1))+
  geom_function(aes(color="Lognormal\nmean=1, sdlog=0.5"),
    fun=dlnorm,
    args=list(meanlog = log(1), sdlog = 0.2))+
  geom_function(aes(color="Lognormal\nmean=1, sdlog=0.2"),
    fun=dlnorm,
    args=list(meanlog = log(1), sdlog = 0.5))+
  ggtitle("Priors for starting biomass ratio B0/K with median of 1")+
  labs(x="phi=B0/K",y="Probability",color="")
```



For small values of log-sd, the log-sd is very similar to the CV which is $\text{mean}(x)/\text{sd}(x)$, so they are sometimes used interchangeably.

Using JABBA

First, install JABBA and JAGS

```
# Installation
# install.packages(devtools)
# devtools::install_github("jabbamodel/JABBA")
```

Original South Atlantic Swordfish example Winker et al. (2018). JABBA: Just Another Biomass Assessment

```
library(JABBA)
swos <- iccat$swos

assessment <- "SWOSiccat"
scenario <- "Schaefer"

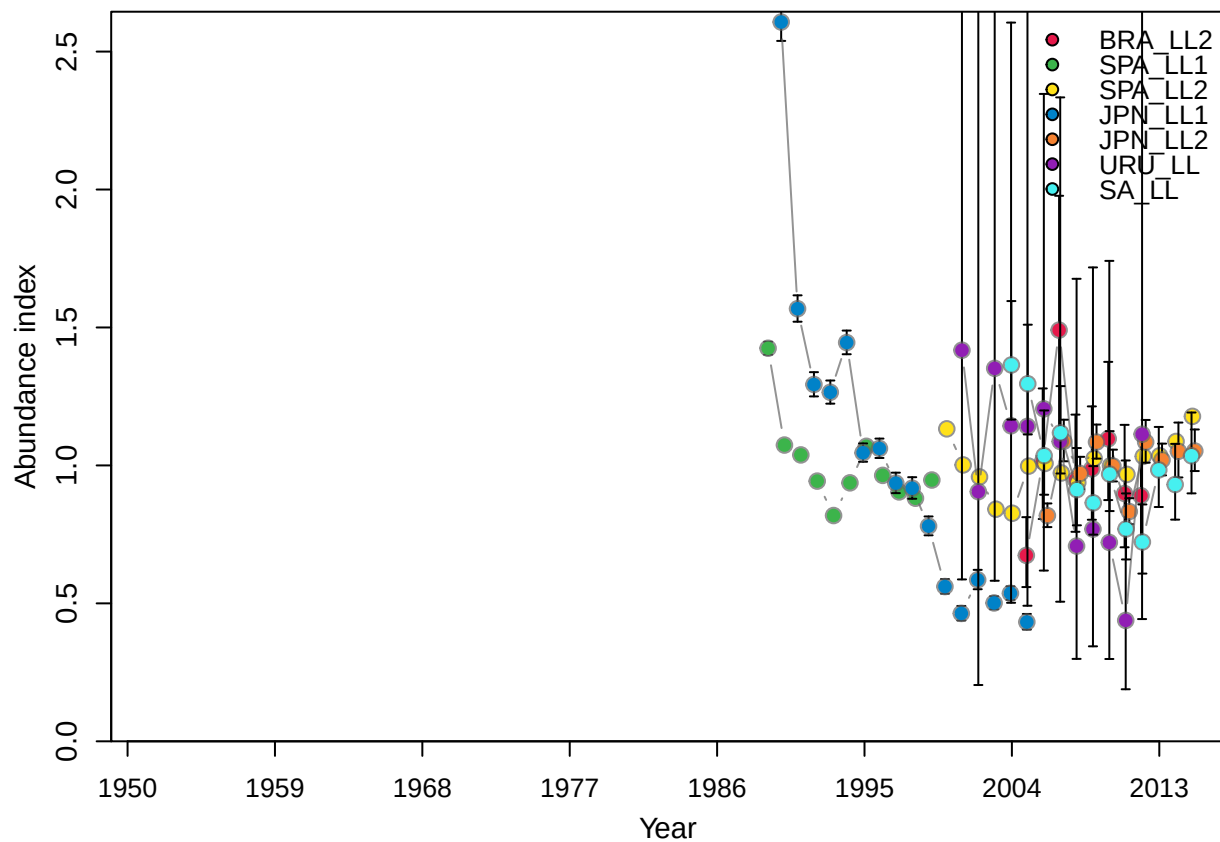
File <- getwd() # "C:/Users/Elizabeth A Babcock/Box/RSM 612/RSM 612 lectures" # LINK to your folder of c
assessment <- "SW0"
output.dir = file.path(File, assessment)
dir.create(output.dir, showWarnings = FALSE)
```

Compile JABBA JAGS model and input object

```
jbswos <- build_jabba(catch=swos$catch, #vector of catches
  cpue=swos$cpue, #data frame of CPUE by fleet
  se=swos$se, #input standard deviations.
  assessment=assessment, #name of assessment
  scenario = scenario, #scenario name
  model.type = "Schaefer", #Schaefer is the logistic model
  r.prior=c(0.42,0.37), #mean and log-sd for lognormal
  K.prior = c(250000,1), #mean and log-sd for lognormal
  psi.prior = c(1,0.25), #mean and log-sd for lognormal
  sigma.est = TRUE, # observation error by fleet
  igamma=c(4,0.01), # default process error
  P_bound = c(0.02,1.1),
  verbose=FALSE, #Bounds of B/K
  sigmaobs_bound = 0.1, #upper limit of observation error
  sigmaproc_bound = 0.2) #upper limit of process error)
```

Plot indices

```
jbplot_indices(jbswos)
```



fit JABBA

This is running a model in JAGS so it takes some time


```
fit.swos <- fit_jabba(jbswos, #the build_jabba object
                     save.jabba=TRUE, #TO save the directory
                     output.dir=output.dir,
                     ni = 50000, #ni,nt,nb and nc are MCMC settings
                     nt = 10,
                     nb = 5000,
                     nc = 2)
```

```
## module glm loaded
```

```
## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 226
##   Unobserved stochastic nodes: 990
##   Total graph size: 6290
##
## Initializing model
```

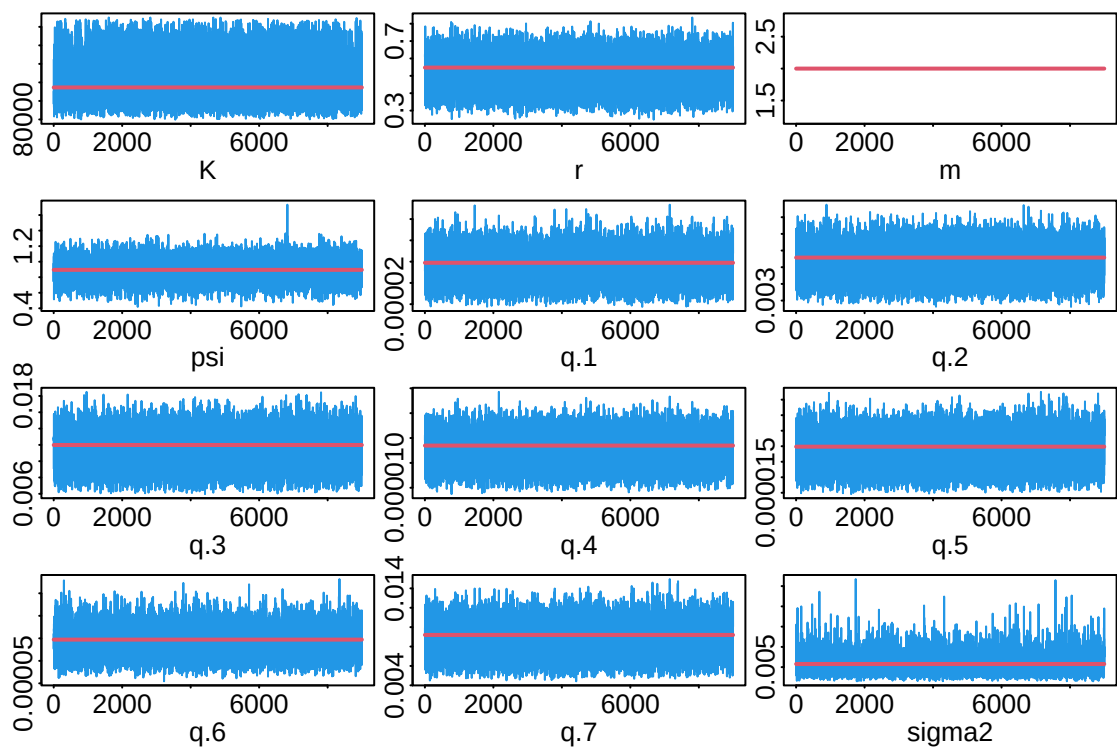
```
##
## ><> Produce results output of Schaefer model for SWO Schaefer <><
```

```
##
##
## ><> Scenario Schaefer_Schaefer completed in 6 min and 67 sec <><
```

Plots to see if the MCMC converged

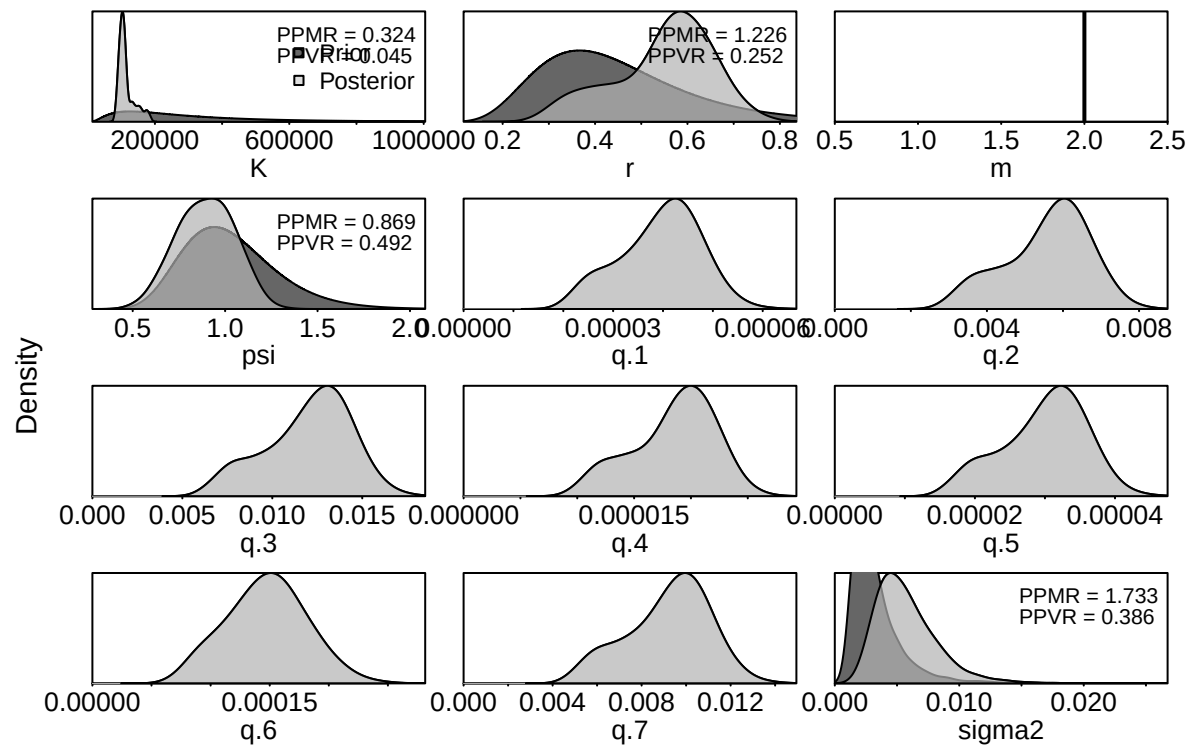
```
jbplot_mcmc(fit.swos)
```

```
##
## ><> jbplot_mcmc() - mcmc chains <><
```



```
jplot_ppdist(fit.swos)  #Posterior and prior distributions
```

```
##
## >< jplot_ppist() - prior and posterior distributions <><
```

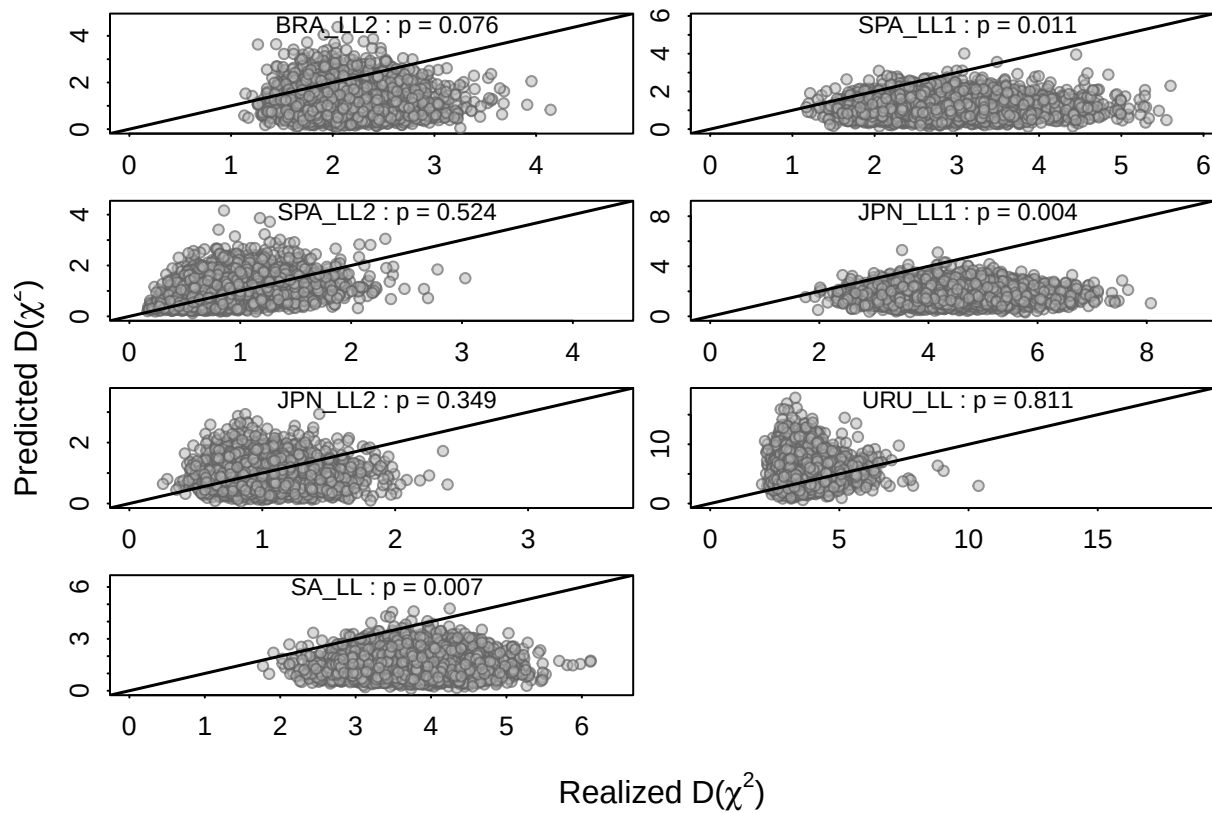


```
jrplot_PPC(fit.swos) #Posterior predictive checks
```

```
##
## >< jrplot_PPC() - Posterior Predictive Checks ><

##
## Posterior Predictive Checks with Bayesian p values

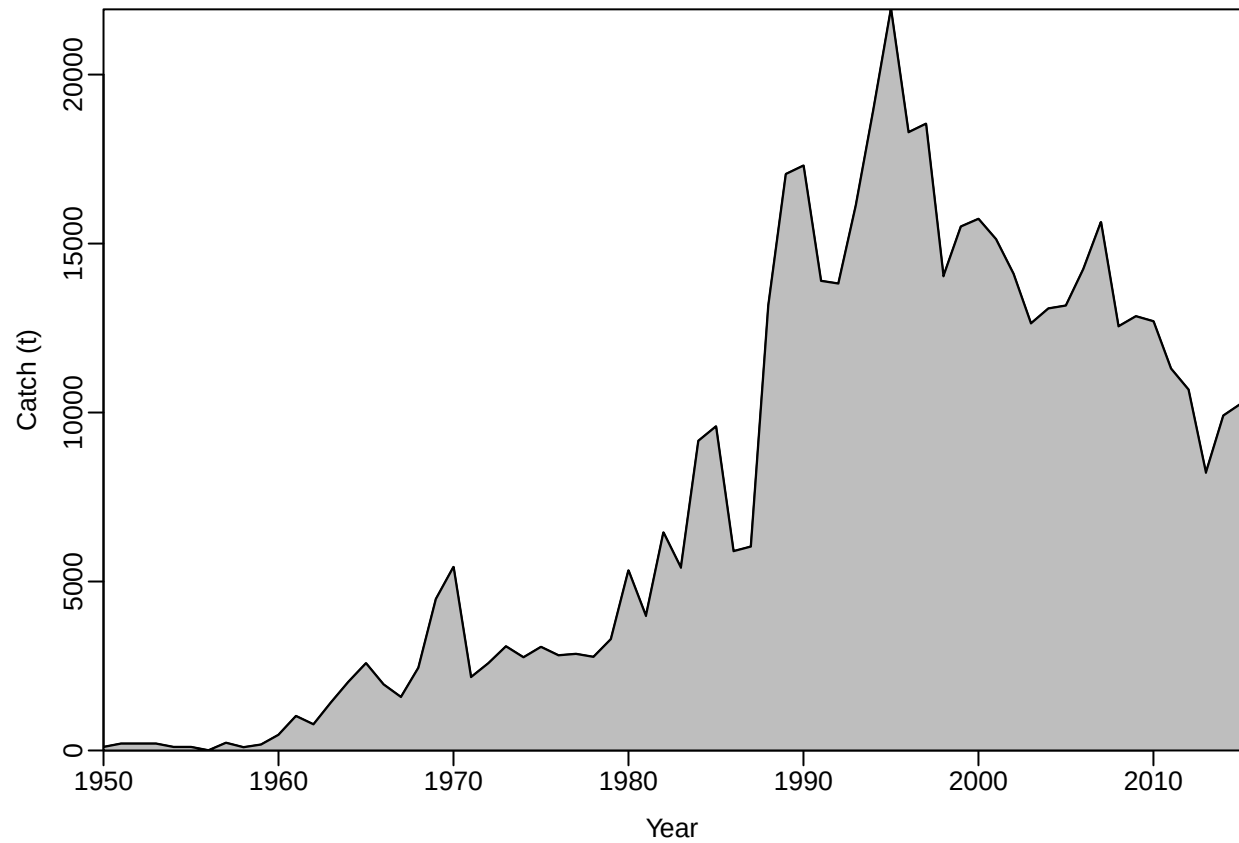
##      Index  Bayesian.p  nobs
## 1  BRA_LL2 0.075777778    8
## 2  SPA_LL1 0.010888889   11
## 3  SPA_LL2 0.524333333   16
## 4  JPN_LL1 0.004444444   16
## 5  JPN_LL2 0.349444444   10
## 6  URU_LL  0.811333333   12
## 7   SA_LL  0.006888889   12
## 8 Combined 0.254730159   85
```



Make some plots to see if the fit is good

```
jbplot_catch(fit.swos) #catches
```

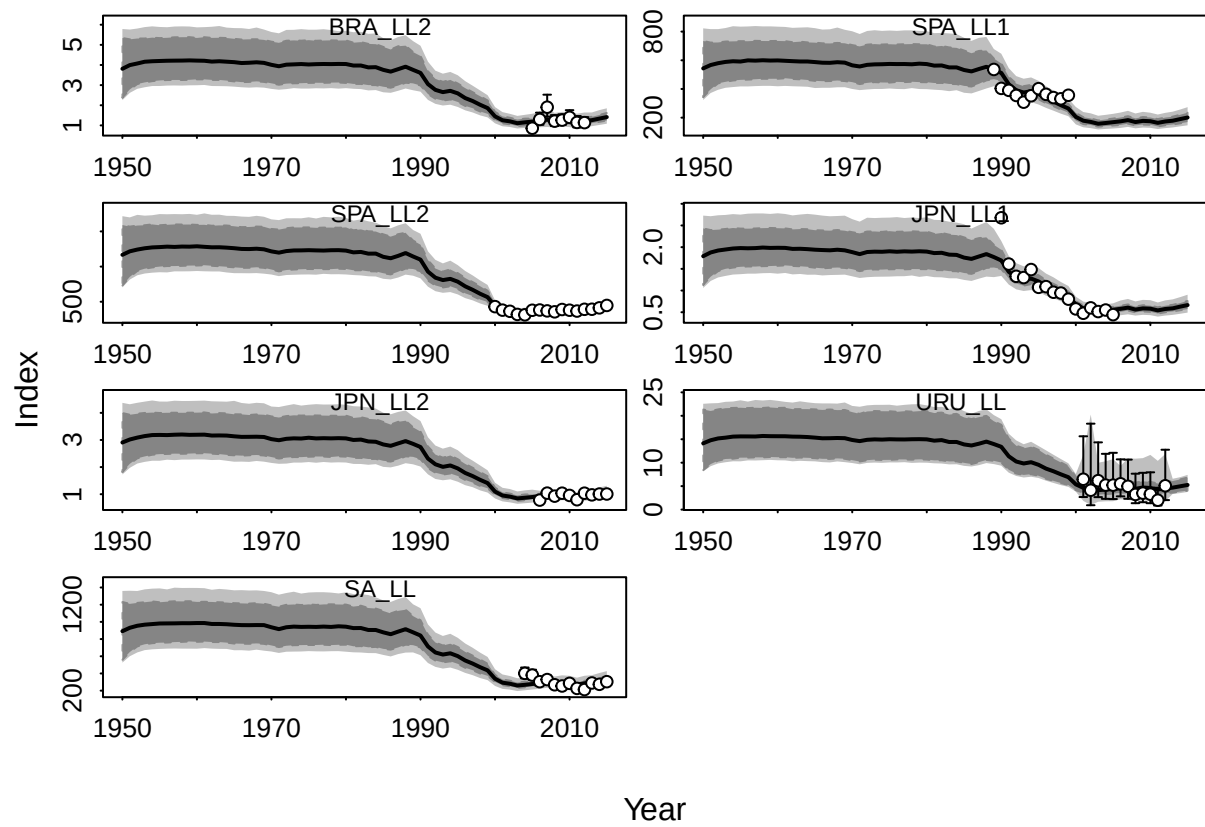
```
##
## >< jbplot_catch() <><
```



```
jbplot_cpuefits(fit.swos) #Fits to CPUE indices
```

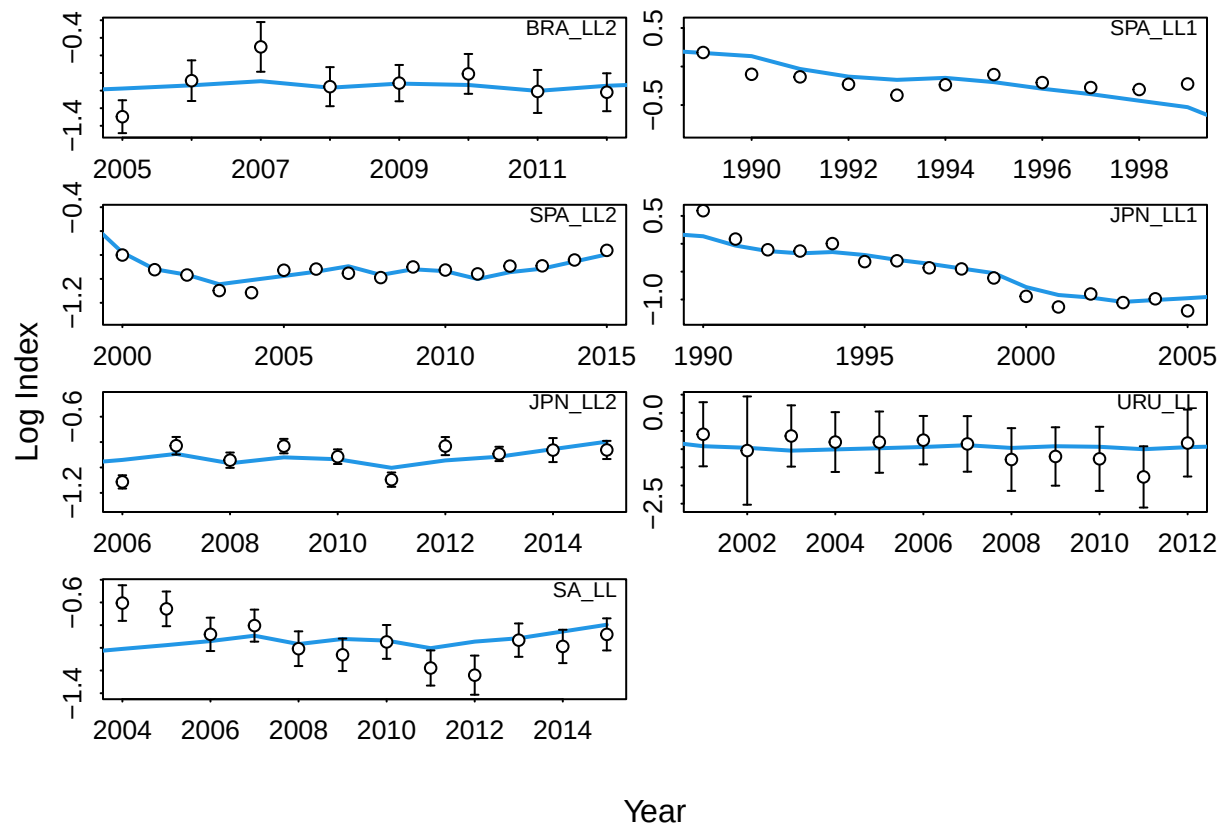
```
##  
## ><> jbplot_cpue() - fits to CPUE <><
```

```
jbplot_logfits(fit.swos) #Fits to CPUE indices in log scale
```

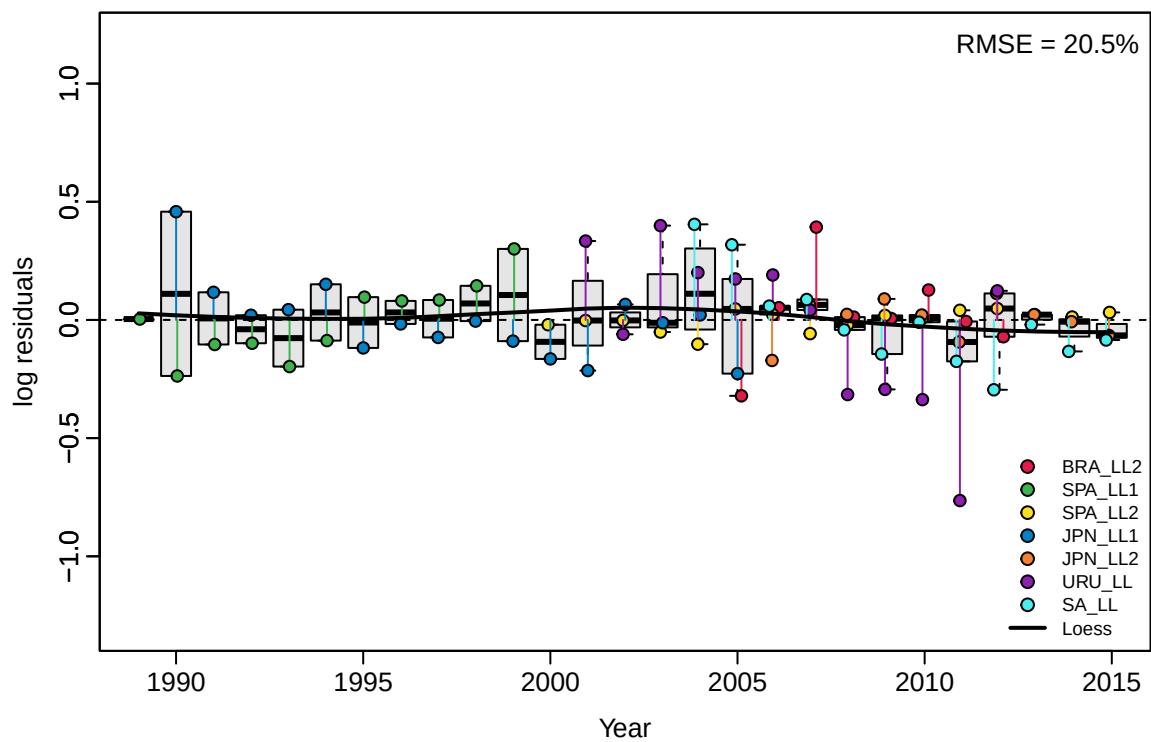


```
##
## ><> jbplot_cpue() - fits to CPUE <><
```

```
jbplot_residuals(fit.swos) # Residuals
```

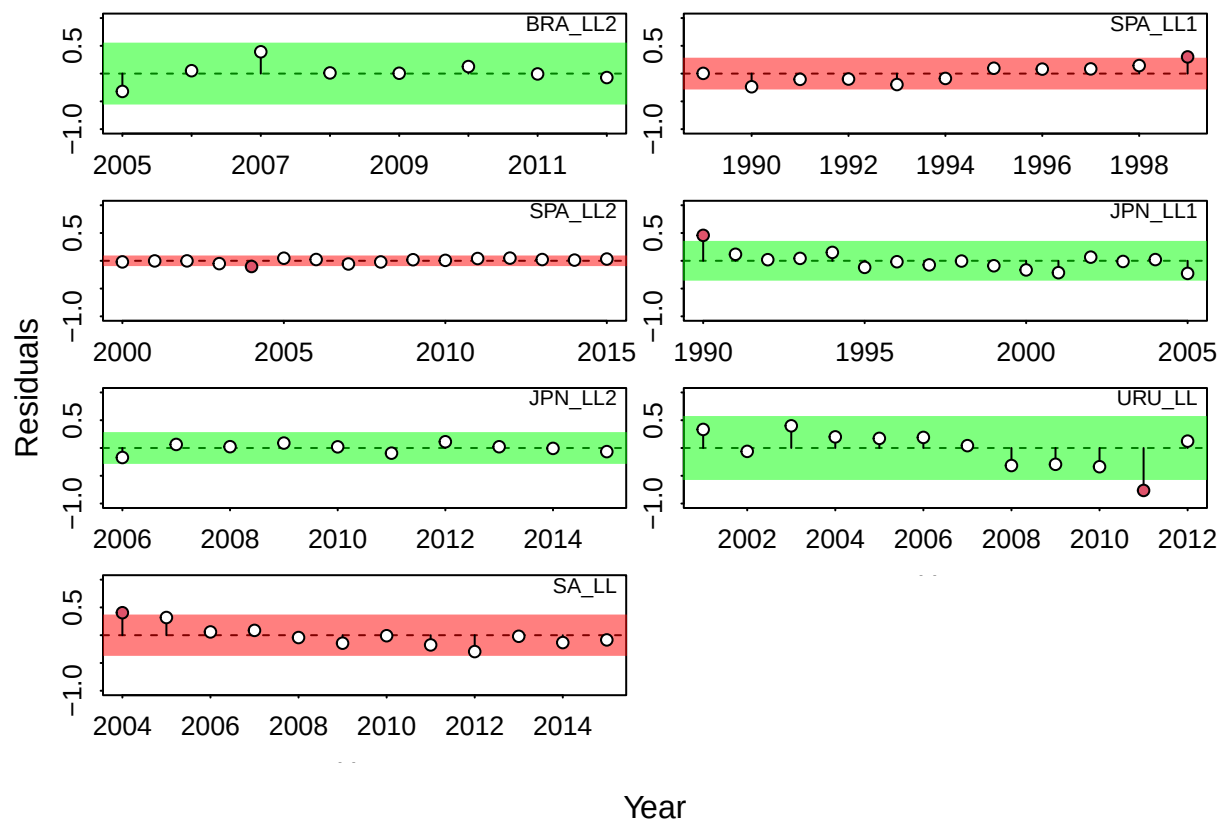


```
##
## ><> jplot_cpue() - fits to CPUE <><
```



```
jbplot_runstest(fit.swos) # A test of whether there are patterns in residuals
```

```
##
## >< jbplot_runstest() <><
```

Plots to see the status of the population

```
jbplot_ensemble(fit.swos,subplots=1:2)
jbplot_kobe(fit.swos)
```

```
##
## ><> jbplot_kobe() - Stock Status Plot <><
```

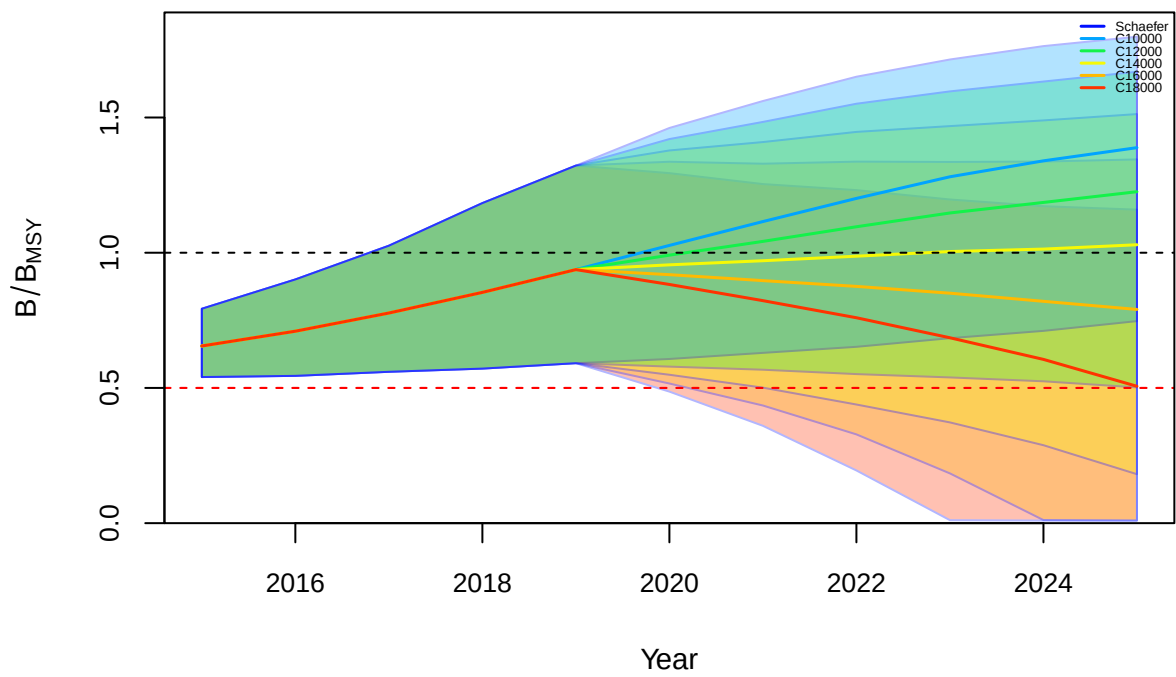
```
jbplot_spphase(fit.swos)
```

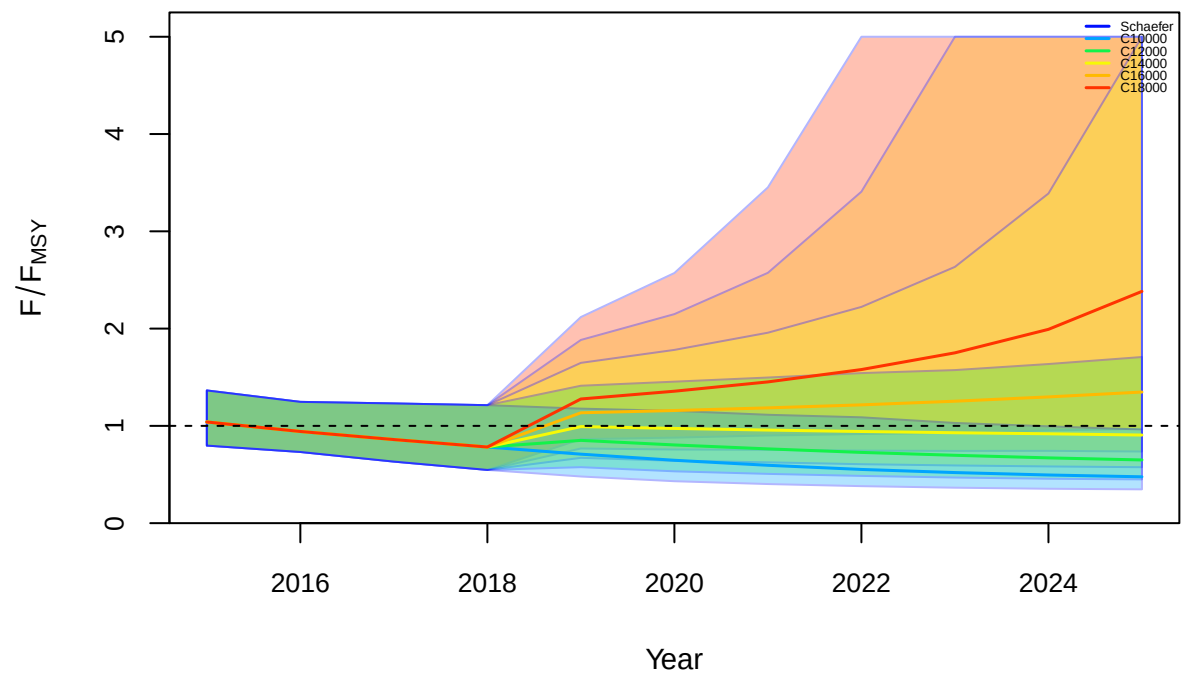
```
##
## ><> jbplot_spphase() - JABBA Surplus Production Phase Plot <><
```

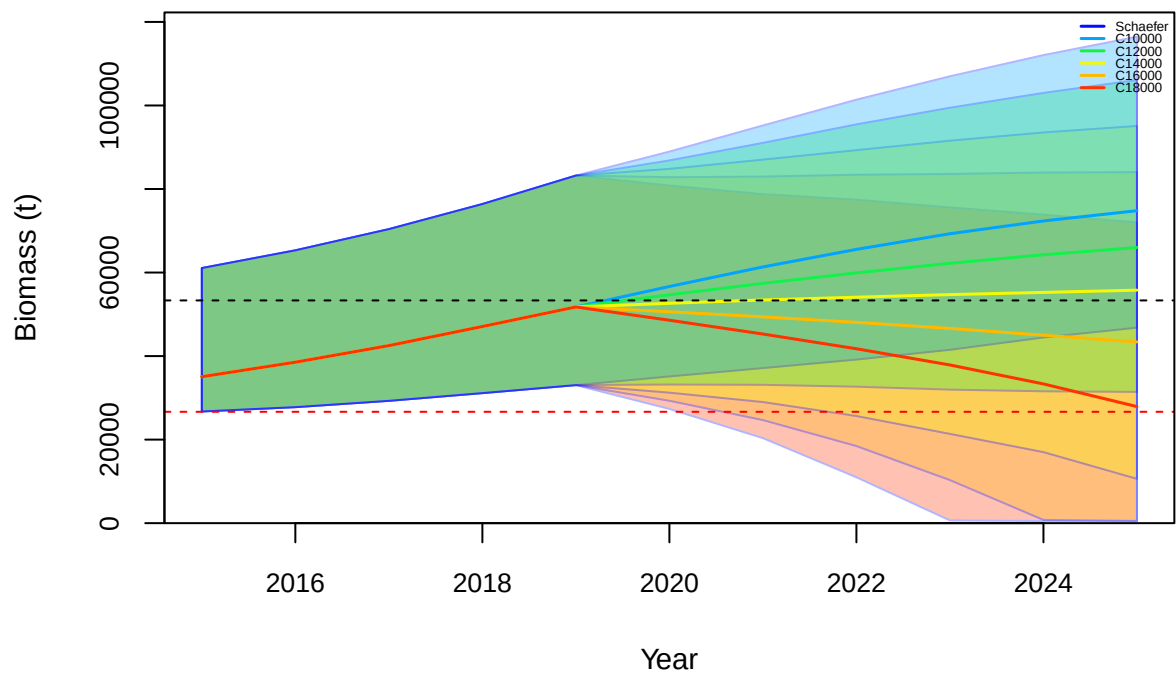
Projections

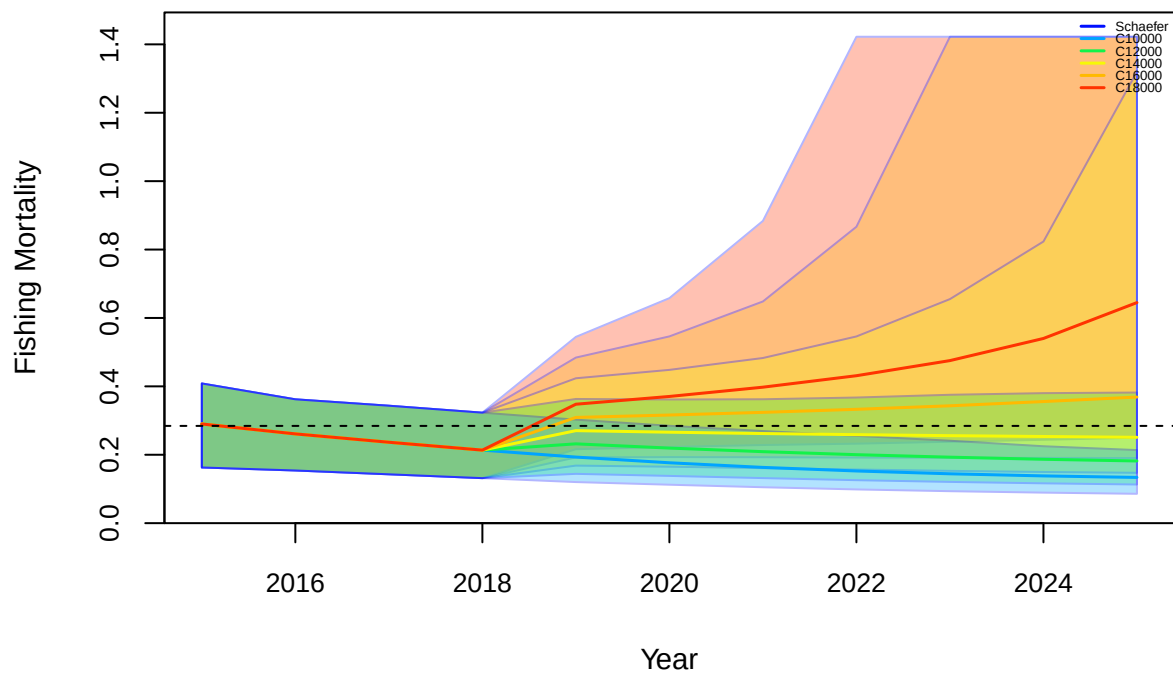
ICCAT Style projections, based on a TAC: total allowable catch

```
Ccur = 10056
TACs = seq(10000,18000,2000)
fw.swos= fw_jabba(fit.swos,nyears=10,imp.yr=4,
                  initial = Ccur,
                  imp.values = TACs,
                  quant="Catch",type="abs",
                  nsq=3,stochastic = T,AR1=F)
jbplot_ensemble(fw.swos,legendcex = 0.4,xlim=c(2015,2025),subplots = c(1:4),add=T)
```





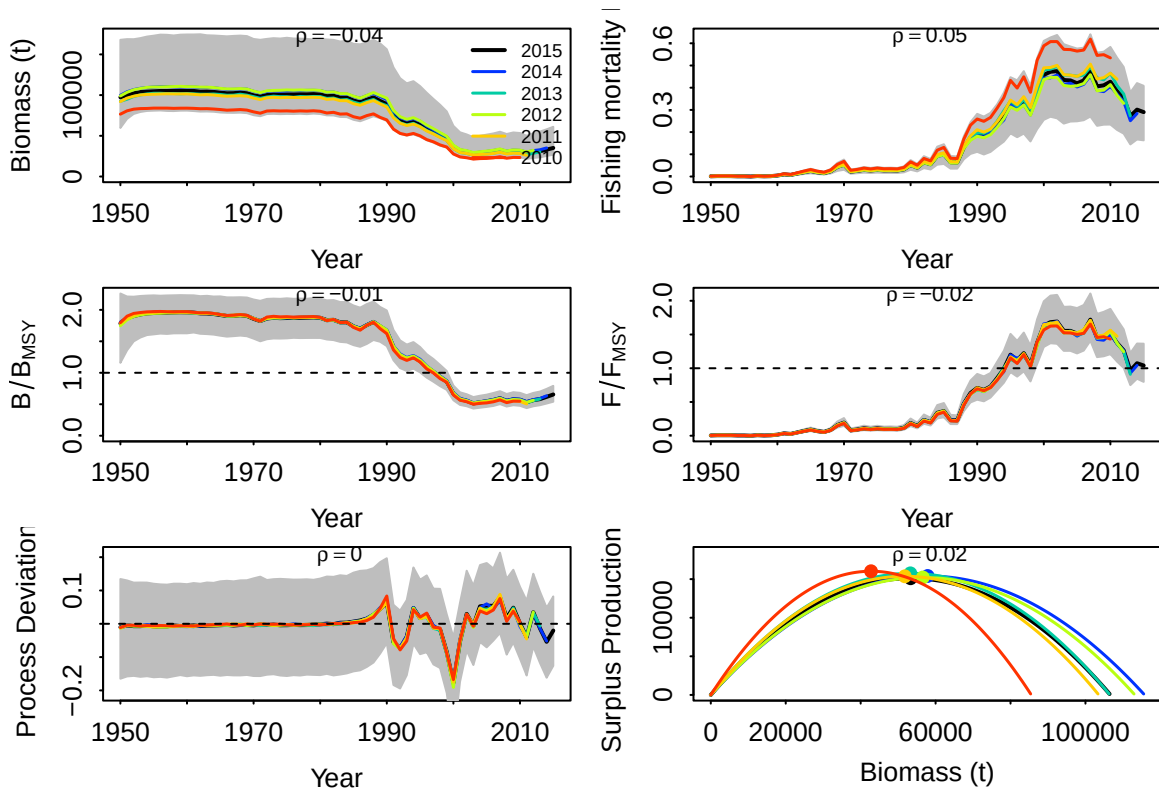




Hindcast cross-validation and retrospective analysis

```
hc1 <- hindcast_jabba(jbswos,fit.swos,peels=1:5)
# Show Retrospective Pattern
jbplot_retro(hc1)
```

```
##
## ><> jbplot_retro() - retrospective analysis <><
```



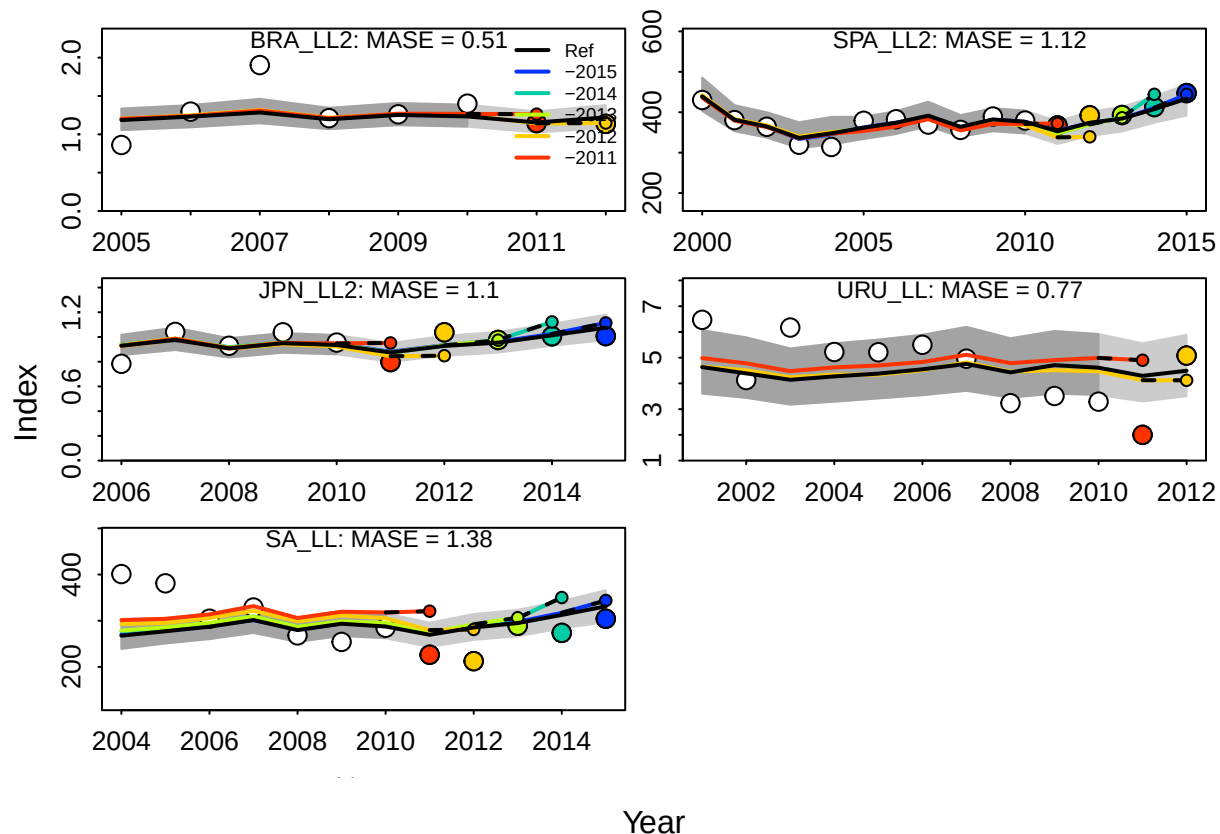
```
##           B           F           Bmsy           Fmsy           procB
## 2015  0.06331056 -0.057712989  0.026847308 -0.03037881  0.0023464341
## 2014 -0.03266532  0.002394098  0.003102544 -0.05808556  0.0070157129
## 2013  0.06422521 -0.069060108 -0.002844063 -0.01551452  0.0004132626
## 2012 -0.08315865  0.139381127 -0.052558598  0.07217953 -0.0055057080
## 2011 -0.23182785  0.251658125 -0.038835682 -0.05768304  0.0051419451
## rho.mu -0.04402321  0.053332051 -0.012857698 -0.01789648  0.0018823293
##           MSY
## 2015  0.001337142
## 2014  0.026769865
## 2013  0.010537331
## 2012  0.018672630
## 2011  0.060790984
## rho.mu 0.023621591
```

```
jbplot_hcxval(hc1)
```

```
##
## ><> Only including indices that have years overlapping hind-cast horizon
##
## ><> Including indices: BRA_LL2 SPA_LL2 JPN_LL2 URU_LL SA_LL
##
## Computing MASE with only 2 of 5 prediction residuals for Index BRA_LL2
##
## Warning: Unequal spacing of naive predictions residuals may influence the interpretation of MASE
```

```
##
##
## No observations in evaluation years to compute prediction residuals for Index SPA_LL1
##
## Computing MASE with all 5 of 5 prediction residuals for Index SPA_LL2
##
## No observations in evaluation years to compute prediction residuals for Index JPN_LL1
##
##Computing MASE with all 5 of 5 prediction residuals for Index JPN_LL2
##
##Computing MASE with only 2 of 5 prediction residuals for Index URU_LL
##
##Warning: Unequal spacing of naive predictions residuals may influence the interpretation of MASE
##
##
##Computing MASE with all 5 of 5 prediction residuals for Index SA_LL
```

##	Index	MASE	MASE.adj	MAE.PR	MAE.base	n.eval
## 1	BRA_LL2	0.5087284	0.3541842	0.05305869	0.10429670	2
## 2	SPA_LL1	NA	NA	NA	NA	0
## 3	SPA_LL2	1.1222403	0.5137691	0.05137691	0.04578067	5
## 4	JPN_LL1	NA	NA	NA	NA	0
## 5	JPN_LL2	1.0979367	0.7926831	0.11811819	0.10758197	5
## 6	URU_LL	0.7730865	0.7730865	0.55271995	0.71495223	2
## 7	SA_LL	1.3835771	1.2502502	0.21123200	0.15267092	5
## 8	joint	0.9831245	0.9831245	0.16395752	0.16677188	19



Pella-Tomlinson model

```
scenario<-"Pella"
jbswos <- build_jabba(catch=swos$catch, #vector of catches
                     cpue=swos$cpue, #data frame of CPUE by fleet
                     se=swos$se, #input standard deviations.
                     assessment=assessment, #name of assessment
                     scenario = scenario, #scenario name
                     model.type = "Pella", #Pella is an assymetric yield curve,
                     BmsyK = 0.4, #Location of Bmsy/K
                     r.prior=c(0.42,0.37), #mean and log-sd for lognormal
                     K.prior = c(250000,1), #mean and log-sd for lognormal
                     psi.prior = c(1,0.25), #mean and log-sd for lognormal
                     sigma.est = TRUE, # observation error by fleet
                     igamma=c(4,0.01), # default process error
                     P_bound = c(0.02,1.1), #Bounds of B/K
                     sigmaobs_bound = 0.1, #upper limit of observation error
                     sigmaproc_bound = 0.2) #upper limit of process error)
```

##

><> Prepare JABBA input data <><

##

><> Assume Catch with error CV = 0.1 <><

##

><> Model type:Pella <><

##

><> Shape m =1.188

##

><> K prior mean =250000and CV =1(log.sd = 0.832554611157698)

##

><> r prior mean =0.42and CV =0.383031945716667(log.sd = 0.37)

##

><> Psi (B1/K) prior mean =1and CV =0.25withlnormdeistribution

##

##

##

><> ALWAYS ENSURE to adjust default settings to your specific stock <><

```
fit.swos2 <- fit_jabba(jbswos, #the build_jabba object
                      save.jabba=TRUE, #TO save the directory
                      output.dir=output.dir,
                      ni = 50000, #ni,nt,nb and nc are MCMC settings
                      nt = 10,
                      nb = 5000,
                      nc = 2)
```



```
## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 226
##   Unobserved stochastic nodes: 990
##   Total graph size: 6290
##
## Initializing model

##
## ><> Produce results output of Pella model for SW0 Pella <><

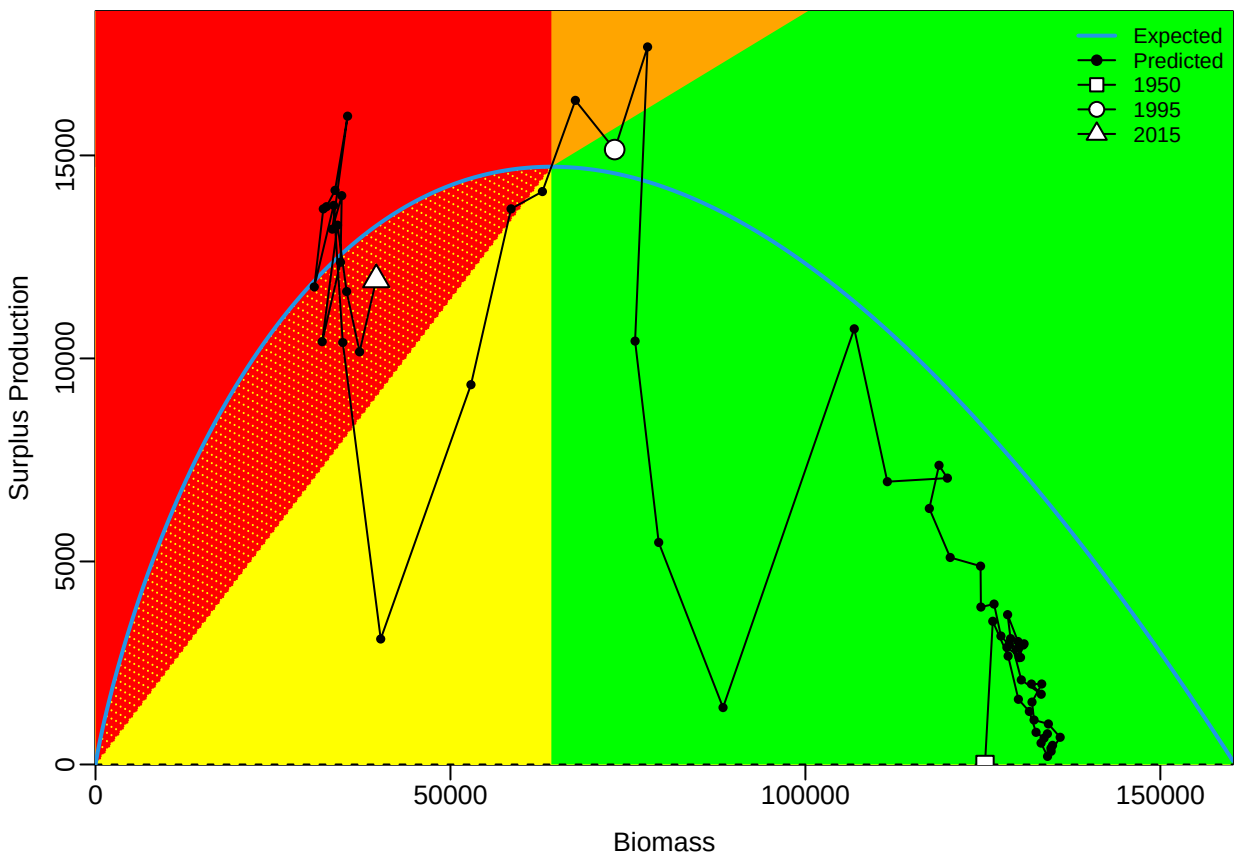
##
##
## ><> Scenario Pella_Pella completed in 13 min and 86 sec <><
```

```
#save.image()
```

Plot yield curve with status

```
jbplot_spdyn(fit.swos2)
```

```
##
## ><> jbplot_spdyn() -  $S_{Pt+1} = B_{t+1} - B_t + C_t$  vs  $B$  <><
```



Compare runs

```
jbplot_ensemble(list(fit.swos,fit.swos2),subplots=1:2)  
jbplot_kobe(fit.swos2)
```

##

<> jbplot_kobe() - Stock Status Plot <><